



Data Wrangling in R: Part 1

Importing Data and Variable Manipulation with dplyr

Siyabonga Mavundla

South African Medical Research Council (SAMRC)

Introduction to R: Pre-Workshop Course



Learning Objectives

By the end of this session, you will be able to:

- ▶ Install and load R packages
- ▶ Import data from different file formats
- ▶ Export data into different file formats
- ▶ Explore and inspect datasets
- ▶ Select variables using `select()`
- ▶ Filter observations using `filter()`
- ▶ Create variables using `mutate()`
- ▶ Sort data using `arrange()`
- ▶ Group data using `group_by()`
- ▶ Produce summaries using `summarise()`

Goal: Move from raw data to analysis-ready data.

1. Packages

Install packages once, then load them whenever you use R.

Install → Load → Use

Installing Packages

Packages only need to be installed once.

```
install.packages("dplyr")  
install.packages("readr")  
install.packages("readxl")  
install.packages("haven")  
install.packages("writexl")
```

What do these packages do?

- ▶ dplyr – Data manipulation
- ▶ readr – Import and export CSV
- ▶ readxl – Import Excel files
- ▶ haven – Stata, SPSS and SAS
- ▶ writexl – Export Excel files

Loading Packages

Packages must be loaded every time you start R.

```
library(dplyr)
library(readr)
library(readxl)
library(haven)
library(writexl)
```

Why load libraries?

Loading a package makes its functions available for use in your R session.

2. Importing Data

Bring data from external files into R.

File → R Dataset → Analysis

Common Data Sources

In research, data are commonly stored in:

- ▶ CSV files (.csv)
- ▶ Excel files (.xlsx)
- ▶ Stata datasets (.dta)
- ▶ SPSS datasets (.sav)
- ▶ SAS datasets (.sas7bdat)
- ▶ Text files (.txt)

Before analysis: data must be imported into R.

What If the Dataset Has No Column Names?

Sometimes a dataset is provided without variable names.

Example: Raw dataset

1001	Female	25	Durban	120
1002	Male	32	Durban	850
1003	Female	41	PMB	1500
1004	Male	29	Durban	300
1005	Female	37	Richards Bay	90

Notice

There are **no column names**; the first row contains actual data.

Importing Data Without Column Names

```
patients <- read_csv(  
  "patients.csv",  
  col_names = FALSE  
)
```

Add meaningful column names:

```
names(patients) <- c(  
  "PatientID", "Sex", "Age",  
  "Site", "ViralLoad"  
)
```

`col_names = FALSE` → **First row is data** → **Add column names**

Importing a CSV File

```
library(readr)

patients <- read_csv(
  "samrc_patients.csv"
)
```

Why CSV?

CSV files are widely used because they can be read by almost any statistical software.

Tip: Use a simple relative file path when possible.

Importing an Excel File

```
library(readxl)

patients <- read_excel(
  "samrc_patients.xlsx"
)
```

Import a specific sheet:

```
patients <- read_excel(
  "samrc_patients.xlsx",
  sheet = "Patients"
)
```

Importing a Stata Dataset

```
library(haven)

patients <- read_dta(
  "samrc_patients.dta"
)
```

Stata files are common in public health and epidemiological research.

Importing an SPSS Dataset

```
library(haven)

patients <- read_sav(
  "samrc_patients.sav"
)
```

SPSS files are frequently used for surveys and questionnaire-based studies.

Importing a SAS Dataset

```
library(haven)

patients <- read_sas(
  "samrc_patients.sas7bdat"
)
```

SAS files are common in clinical trials and pharmaceutical studies.

Importing Data: Quick Reference

File	Function	Package
CSV	<code>read_csv()</code>	readr
Excel	<code>read_excel()</code>	readxl
Stata	<code>read_dta()</code>	haven
SPSS	<code>read_sav()</code>	haven
SAS	<code>read_sas()</code>	haven

Remember

The function you use depends on the format of your data file.

Inspecting Imported Data

Always inspect your data before analysis.

```
head(patients)
dim(patients)
str(patients)
summary(patients)
```

Function	Purpose
head()	First six records
dim()	Number of rows and columns
str()	Data structure and variable types
summary()	Descriptive statistics

3. Exporting Data

Save R datasets to external files.

R Dataset → External File → Share / Store

Why Export Data?

After cleaning and manipulating data in R, you may need to:

- ▶ Share the dataset with collaborators
- ▶ Open the dataset in another software package
- ▶ Save a cleaned version of the data
- ▶ Create a file for reporting or further analysis

Import brings data into R.
Export saves data from R to a file.

Packages for Exporting Data

Some export functions require additional packages.

```
install.packages("writexl")  
install.packages("haven")  
install.packages("readr")
```

- ▶ readr – CSV
- ▶ writexl – Excel

- ▶ haven – Stata, SPSS, SAS

Exporting a CSV File

Use `write_csv()` from the `readr` package.

```
library(readr)

write_csv(
  patients,
  "patients_export.csv"
)
```

CSV is a good choice when you want a simple and widely compatible data file.

Exporting an Excel File

Use `write_xlsx()` from the `writexl` package.

```
library(writexl)

write_xlsx(
  patients,
  "patients_export.xlsx"
)
```

Excel files are commonly used for storing, sharing, and exchanging structured data.

Exporting a Stata Dataset

Use `write_dta()` from the `haven` package.

```
library(haven)

write_dta(
  patients,
  "patients_export.dta"
)
```

Useful when your collaborators or analysis workflow uses Stata.

Exporting an SPSS Dataset

Use `write_sav()` from the `haven` package.

```
library(haven)

write_sav(
  patients,
  "patients_export.sav"
)
```

Exporting a SAS Dataset

Use `write_sas()` from the `haven` package.

```
library(haven)

write_sas(
  patients,
  "patients_export.sas7bdat"
)
```

4. Our Working Dataset

We will use a small patient dataset throughout the session.

Creating the Example Dataset

```
patients <- data.frame(  
  PatientID = c(1001,1002,1003,1004,1005),  
  Sex = c("Female","Male","Female",  
          "Male","Female"),  
  Age = c(25,32,41,29,37),  
  Site = c("Durban","Durban","PMB",  
           "Durban","Richards Bay"),  
  ViralLoad = c(120,850,1500,300,90)  
)
```

Our Working Dataset

PatientID	Sex	Age	Site	ViralLoad
1001	Female	25	Durban	120
1002	Male	32	Durban	850
1003	Female	41	PMB	1500
1004	Male	29	Durban	300
1005	Female	37	Richards Bay	90

Variables

- ▶ PatientID
- ▶ Sex
- ▶ Age

Variables

- ▶ Site
- ▶ ViralLoad

Understanding the Variables

Variable	Description
PatientID	Unique patient identifier
Sex	Gender of the participant
Age	Participant age in years
Site	Clinic or study site
ViralLoad	HIV viral load measurement

Important: Rows represent observations; columns represent variables.

5. Data Manipulation with dplyr

Six core functions for everyday data wrangling

`select()`

`filter()`

`mutate()`

`arrange()`

`group_by()`

`summarise()`

select(): Choosing Variables

select() chooses specific variables (columns) from a dataset.

Suppose we only need:

- ▶ Patient ID
- ▶ Age
- ▶ Viral Load

```
patients %>%  
  select(  
    PatientID,  
    Age,  
    ViralLoad  
  )
```

The Pipe Operator: %>%

Think of %>% as the word:

“then”

```
patients %>%  
  select(PatientID, Age, ViralLoad)
```

Read this as:

Go into the “patients” dataset, and then select PatientID, Age, and ViralLoad.

The pipe makes a sequence of data-wrangling steps easier to read.

filter(): Choosing Observations

filter() selects rows that meet specified conditions.

Patients aged 30 years and older:

```
patients %>%  
  filter(Age >= 30)
```

Female participants only:

```
patients %>%  
  filter(Sex == "Female")
```

Key idea

filter() works with **rows**.

filter(): Multiple Conditions

Female participants older than 30 years:

```
patients %>%  
  filter(  
    Sex == "Female",  
    Age > 30  
  )
```

Result:

PatientID	Sex	Age	Site	ViralLoad
1003	Female	41	PMB	1500
1005	Female	37	Richards Bay	90

mutate(): Creating New Variables

mutate() creates new variables or modifies existing variables.

Create viral suppression status:

```
patients %>%  
  mutate(  
    Status = ifelse(  
      ViralLoad < 1000,  
      "Suppressed",  
      "Unsuppressed"  
    )  
  )
```

A new variable called Status is added.

mutate(): Before and After

Before	
PatientID	ViralLoad
1001	120
1002	850
1003	1500
1004	300
1005	90

After adding Status		
PatientID	ViralLoad	Status
1001	120	Suppressed
1002	850	Suppressed
1003	1500	Unsuppressed
1004	300	Suppressed
1005	90	Suppressed

What happened?

`mutate()` created a new variable called `Status`. Patients with a Viral Load less than 1000 were classified as **Suppressed**, while those with a Viral Load of 1000 or more were classified as **Unsuppressed**.

arrange(): Sorting Data

arrange() sorts rows based on one or more variables.

Youngest to oldest:

```
patients %>%  
  arrange(Age)
```

Oldest to youngest:

```
patients %>%  
  arrange(desc(Age))
```

Use desc() when you want descending order.

group_by(): Creating Groups

group_by() divides a dataset into groups so calculations can be performed separately for each group.

Separate data by sex:

```
patients %>%  
  group_by(Sex)
```

Groups:

- ▶ Female
- ▶ Male

Important

group_by() usually becomes most useful when combined with summarise().

summarise(): Generating Statistics

summarise() calculates summary statistics.

Average age:

```
patients %>%  
  summarise(  
    Mean_Age = mean(Age)  
  )
```

Average viral load:

```
patients %>%  
  summarise(  
    Mean_VL = mean(ViralLoad)  
  )
```

group_by() + summarise()

Average age by sex:

```
patients %>%  
  group_by(Sex) %>%  
  summarise(  
    Mean_Age = mean(Age)  
  )
```

Patient counts by site:

```
patients %>%  
  group_by(Site) %>%  
  summarise(  
    Patients = n()  
  )
```

The Six Core dplyr Functions

Function	Question it answers
<code>select()</code>	Which columns do I need?
<code>filter()</code>	Which rows do I need?
<code>mutate()</code>	What new variable do I need?
<code>arrange()</code>	How should the rows be ordered?
<code>group_by()</code>	How should the data be divided into groups?
<code>summarise()</code>	What statistic do I want?

These functions can be combined to build powerful data-wrangling workflows.

Complete Data Wrangling Workflow

```
patients %>%  
  filter(Age >= 30) %>%  
  mutate(  
    Status = ifelse(  
      ViralLoad < 1000,  
      "Suppressed",  
      "Unsuppressed"  
    )  
  ) %>%  
  group_by(Site) %>%  
  summarise(  
    Patients = n(),  
    Mean_Age = mean(Age)  
  ) %>%  
  arrange(desc(Patients))
```

Reading the Complete Pipeline

The code performs the following steps:

1. **Filter** patients aged 30 years and older.
2. **Create** viral suppression status.
3. **Group** patients by study site.
4. **Calculate** the number of patients per site.
5. **Calculate** mean age per site.
6. **Sort** sites from largest to smallest patient count.

Each step takes the result from the previous step.

Practical Example

Using the `patients` dataset:

1. Select `PatientID`, `Age`, and `Site`.
2. Keep only patients aged 30 or older.
3. Create a variable called `Status`.
4. Sort patients from oldest to youngest.
5. Calculate the average age by site.
6. Count the number of patients at each site.

Example Solution

Example solution:

```
patients %>%  
  filter(Age >= 30) %>%  
  group_by(Site) %>%  
  summarise(  
    Patients = n(),  
    Mean_Age = mean(Age)  
  ) %>%  
  arrange(desc(Patients))
```

There may be more than one correct solution.

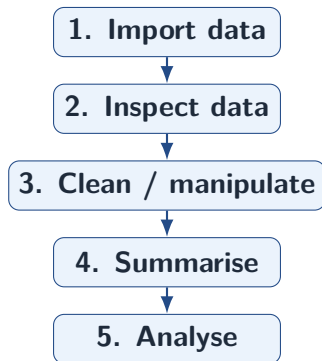
Common Mistakes

- ▶ Forgetting to load the required package
- ▶ Using = instead of == in a condition
- ▶ Mixing up rows and columns
- ▶ Misspelling variable names
- ▶ Forgetting quotation marks around text values
- ▶ Forgetting the pipe between steps
- ▶ Using the wrong import function for the file type

Tip

Read error messages carefully. R often tells you where the problem is.

A Good Data-Wrangling Workflow



Key Takeaways

- ▶ Import data using `readr`, `readxl` and `haven`
- ▶ Export data using `writexl`, `readr`, and `haven`
- ▶ Use `select()` to choose columns
- ▶ Use `filter()` to choose rows
- ▶ Use `mutate()` to create variables
- ▶ Use `arrange()` to sort data
- ▶ Use `group_by()` to create groups
- ▶ Use `summarise()` to produce statistics

Thank You

Import → Inspect → Manipulate → Summarise → Analyse